

University of Science
Faculty of Information Technology

EduFlex

Design and Implementation of a Mobile Learning Platform

Student: Vu Thanh Dat
Student ID: 23127346
Supervisor: Dr. Truong Toan Thinh
Submission: [Month Year]

Project Report

Declaration

I declare that this report describes my own project work. Sources used to explain technologies, standards, and external systems are acknowledged in the references. Project-specific results are based on the source code and verification records in the EduFlex repository.

Student signature: _____

Date: _____

Acknowledgements

I would like to thank my supervisor, lecturers, classmates, and testers for their guidance and feedback during the development of this project. Their comments helped shape both the learning experience and the engineering decisions in this report.

Abstract

EduFlex is a full-stack mobile learning platform intended to make structured study accessible from an Android device. Learners can discover and enrol in courses, study lessons, complete quizzes, save progress, earn experience points and badges, maintain streaks, review courses, and obtain assistance grounded in course material. An administration area supports the management of users, courses, lessons, and quiz content.

The mobile client is a native Android application developed in Java with XML layouts and Material components. It communicates with a Spring Boot REST API using Retrofit and JSON Web Tokens. PostgreSQL is the system of record, Flyway manages schema evolution, and jOOQ supports explicit type-safe SQL. Redis provides caching and refresh-token revocation, while RabbitMQ moves embedding generation away from the synchronous request. Semantic search uses local MiniLM embeddings stored with pgvector. Gemini generation is optional and grounded with retrieved lesson content.

Reliability is treated as a functional property. Row locks, conditional state transitions, constraints, and transactions protect progress and rewards from duplicate concurrent requests. Testcontainers verifies these rules against a real PostgreSQL/pgvector database. Docker Compose provides local infrastructure. Prometheus, Grafana, Alertmanager, and exporters provide metrics, dashboards, and alerts. GitHub Actions verifies the backend, monitoring configuration, Android lint, and debug and release builds.

At the recorded revision, 20 backend tests passed, including 18 PostgreSQL integration scenarios. A live HTTP journey passed 26 checks, and 100 simultaneous duplicate completion requests produced one progress row and one 20-XP reward. These results demonstrate tested consistency and integration behavior; they do not establish learning effectiveness, usability, or production-scale performance.

Keywords: mobile learning, Android, Spring Boot, PostgreSQL, Redis, RabbitMQ, pgvector, concurrency, observability, continuous integration.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
List of Abbreviations	ix
1 Introduction	1
1.1 Background	1
1.2 Problem statement	1
1.3 Research and evaluation questions	2
1.4 Objectives	2
1.5 Scope and limitations	3
1.6 Report organization	3
2 Related Work and Design Position	4
2.1 Mobile learning	4
2.2 Gamification	4
2.3 Retrieval-grounded assistance	4
2.4 Project contribution	5
3 Requirements Analysis	6
3.1 Actors	6
3.2 Functional requirements	6
3.3 Use cases and acceptance conditions	7
3.4 Non-functional requirements	7
3.5 Requirement prioritization	8
4 Technologies and Tools	9
4.1 Native Android	9
4.2 Spring Boot	9
4.3 Persistence	9

4.4	Caching, messaging, and delivery	10
4.5	Technology selection rationale	10
5	System Architecture and Design	12
5.1	Overview	12
5.2	Backend structure	13
5.3	Android structure	14
5.4	Deployment	14
5.5	Request and failure boundaries	15
6	Database Design	16
6.1	Core model	16
6.2	Ownership and lifecycle	17
6.3	Schema evolution	17
6.4	Integrity and connections	18
6.5	Relational and vector data together	18
7	Mobile Application Implementation	20
7.1	User experience	20
7.2	Networking and authentication	22
7.3	Learning and engagement	22
7.4	UI quality and accessibility considerations	22
7.5	Build variants	23
8	Backend and Security Implementation	24
8.1	REST API	24
8.2	API contract groups	25
8.3	Authentication and authorization	26
8.4	Media	27
8.5	Redis cache policy	28
9	Reliability, Concurrency, and Messaging	29
9.1	Transactional updates	29
9.2	Per-learner serialization	29
9.3	RabbitMQ embeddings	30
9.4	Known retry boundary	31
9.5	Failure analysis	31
10	AI-Assisted Learning	33
10.1	Retrieval	33
10.2	Grounded generation and fallback	33

10.3 Responsible boundaries	34
10.4 Quality evaluation approach	34
11 DevOps, Deployment, and Monitoring	35
11.1 Containerization and configuration	35
11.2 Continuous integration	35
11.3 Metrics, dashboards, and alerts	35
11.4 Operational security	37
11.5 Operational interpretation	37
12 Testing and Evaluation	38
12.1 Strategy	38
12.2 Backend results	40
12.3 End-to-end and Android results	41
12.4 Build and monitoring results	41
12.5 Evaluation limits	41
12.6 Performance claims and missing measurements	42
13 Conclusion and Future Work	43
13.1 Conclusion	43
13.2 Future work	43
13.3 Portfolio summary	44
A Build and Verification Commands	45
B Selected API Groups	46
C Configuration Reference	47
D Diagram and Evidence Index	48
E Submission Checklist	50

List of Figures

5.1	System context and external actors	12
5.2	EduFlex runtime architecture	13
5.3	Backend responsibility layers	13
5.4	Android navigation and principal user journeys	14
5.5	Docker Compose deployment and monitoring topology	15
6.1	Logical database relationship model	17
7.1	End-to-end learner course journey	21
8.1	API groups and authorization boundaries	24
8.2	Login and access-token refresh sequence	27
8.3	Cache-aside read and invalidation flow	28
9.1	Learning-progress state and retry behavior	30
9.2	Concurrency-safe progress transaction	30
9.3	Post-commit content embedding sequence	31
10.1	Grounded course-question flow	34
11.1	Continuous integration order	35
11.2	Metrics, visualization, and alerting flow	36
11.3	Provisioned Grafana Service Overview running against the local stack	36

List of Tables

3.1	Primary actors	6
3.2	Representative use cases and observable completion conditions	7
3.3	Non-functional requirements	8
4.1	Technology choices and project trade-offs	10
6.1	Principal entity groups	16
6.2	Selected integrity rules and their purpose	18
7.1	Implemented learner screens and required visible states	21
8.1	Representative API contracts	25
8.2	Configured Redis cache lifetimes	28
9.1	Reliability mechanisms and remaining boundaries	31
12.1	Evidence mapped to quality goals	39
12.2	Claim-to-evidence traceability	39
12.3	Selected verified scenarios	40
12.4	Required performance experiment design	42

List of Abbreviations

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Delivery
DTO	Data Transfer Object
FCM	Firebase Cloud Messaging
JWT	JSON Web Token
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
TTL	Time To Live
UI/UX	User Interface and User Experience
XP	Experience Points

Chapter 1

Introduction

1.1 Background

Mobile devices make learning material available beyond the classroom, but availability alone does not guarantee effective study. Learners also need a clear path through content, immediate feedback, visible progress, and reasons to return. Mobile learning systems therefore combine content delivery with interaction, assessment, reminders, and progress tracking. Android was selected because it supports a broad range of devices and mature libraries for navigation, background work, networking, and accessibility [5]. A meta-analysis of 110 experimental and quasi-experimental studies reported a moderate average effect for mobile-integrated learning, while showing that outcomes depend on instructional design and context [16]. EduFlex therefore treats the phone as a delivery and interaction channel rather than evidence of educational effectiveness by itself.

EduFlex connects these learner needs to a backend designed for data correctness and operational visibility. The project is also an engineering portfolio: it demonstrates an end-to-end system rather than an isolated interface or collection of endpoints.

1.2 Problem statement

Many prototype learning applications focus on screen design but do not address problems that appear when an action is retried, several requests arrive together, cached data becomes stale, or a dependency fails. One lesson completion can affect progress, XP, quests, course completion, and badges. If these changes are independent, a duplicated request may award a reward twice or leave partially updated data.

The project addresses two connected problems: providing a usable mobile experience for discovering and completing learning content, and preserving trustworthy learning state while supporting caching, asynchronous work, AI assistance, automated testing, and monitoring.

1.3 Research and evaluation questions

The implementation is evaluated through four bounded questions:

1. Can a learner complete the principal account-to-course journey through the Android client and REST API?
2. Do duplicate and concurrent learning commands preserve one authoritative progress transition and one protected reward?
3. Can asynchronous embedding and cache acceleration remain outside the authoritative transaction while failures stay observable?
4. Can a developer reproduce the main build, integration, and monitoring checks from versioned commands and infrastructure definitions?

The report answers these questions with repository inspection and recorded verification. It does not claim that EduFlex improves grades, motivation, or retention because no controlled learner study was conducted.

1.4 Objectives

The main objective is to implement a functional Android learning application backed by a secure and observable service. The specific objectives are:

- provide account, course, lesson, quiz, review, payment-record, and gamification flows;
- present consistent screens with loading, empty, validation, and failure states;
- expose a documented REST API with authentication and role-based access;
- store learning state in a versioned relational schema;
- make progress and rewards safe under retry and concurrency;
- move repeated reads and embedding work into bounded cache and background paths;
- ground optional AI output in stored course material;
- automate build and integration verification; and
- expose application and infrastructure health through dashboards and alerts.

1.5 Scope and limitations

The scope includes a native Android learner application, an Android administration area, Spring Boot, PostgreSQL/pgvector, Redis, RabbitMQ, Docker Compose, tests, GitHub Actions, and a local monitoring stack. Firebase push, Supabase media storage, and Gemini generation are optional integrations. The core development build works without their credentials.

The project does not publish a signed application to Google Play. CI release outputs are unsigned inspection artifacts. Centralized logs, distributed tracing, external alert delivery, and automated database backup are outside the current deployment.

1.6 Report organization

The report moves from requirements and relevant research to implementation and evaluation. It first explains the technology and architectural decisions, then follows data, mobile, API, reliability, AI, and operational concerns. The final chapters map claims to recorded evidence, state evaluation limits, and identify future work.

Chapter 2

Related Work and Design Position

2.1 Mobile learning

Sung, Chang, and Liu synthesized 110 studies and found a moderate overall effect for integrating mobile devices into teaching and learning [16]. Their analysis also identified variation across instructional methods, intervention duration, and implementation settings. This result supports mobile access as a useful design opportunity, but it does not justify assuming that any mobile application improves learning. EduFlex consequently focuses on observable interaction qualities—clear progress, feedback, retry behavior, and access to ordered material—while reserving pedagogical effectiveness for a later learner study.

2.2 Gamification

Gamification research reports generally positive but context-dependent outcomes. Hamari, Koivisto, and Sarsa found that most reviewed empirical studies reported some positive effects, while emphasizing that results depended on the users and setting [8]. EduFlex implements XP, levels, streaks, quests, badges, and a leaderboard as feedback mechanisms. It does not infer improved motivation from their presence. The current evaluation establishes reward consistency; a study of motivation, continued use, and unintended competitive pressure remains necessary.

2.3 Retrieval-grounded assistance

Retrieval-augmented generation combines a generator with an explicit retrievable knowledge source, improving provenance and allowing knowledge to be updated outside model parameters [10]. EduFlex applies the same broad pattern at project scale: it restricts retrieval to one course, ranks lesson material through pgvector, and returns source lessons with the answer. This is an engineering safeguard rather than proof of factual correctness.

Retrieval relevance and answer groundedness require a labelled evaluation set that the present project does not yet contain.

2.4 Project contribution

EduFlex’s contribution is the integration of a native learning client with a backend that treats progress integrity as a first-class requirement. Its strongest evaluated contribution is the transaction boundary joining lesson progress, course completion, XP, quests, and badges under duplicate and concurrent requests. Caching, asynchronous embedding, and monitoring are arranged around that boundary so operational features do not become the source of truth. The project is a full-stack engineering case study, not a controlled study of educational outcomes.

Chapter 3

Requirements Analysis

3.1 Actors

The learner needs a simple path from discovery to completion. The administrator maintains users and content. An operator runs the system and observes its health. External systems optionally provide media storage, remote notifications, and generated text.

Table 3.1: Primary actors

Actor	Responsibilities and goals
Learner	Authenticate, discover courses, enrol, study, complete quizzes, track progress, earn rewards, and review material.
Administrator	Manage users, courses, lessons, and questions through protected operations.
Operator	Configure, deploy, observe, diagnose, and maintain the platform.
External provider	Supply optional storage, notification, or generation services.

3.2 Functional requirements

The account module shall support registration, login, profile update, password recovery, token refresh, and logout. The catalogue shall support listing, categories, search, details, reviews, and learner enrolments. Lessons may contain study content or multiple-choice and fill-in-the-blank quizzes. Progress shall be saved and reflected in continue-learning and course completion.

Gamification shall expose XP, levels, daily check-in, streaks, quests, badges, and a leaderboard. Course assistance shall summarize stored material and answer questions using retrieved context. Administration shall restrict content and user management to an administrator role.

3.3 Use cases and acceptance conditions

Table 3.2: Representative use cases and observable completion conditions

Use case	Main flow	Acceptance condition
Authenticate	Register or log in, then establish a session	Valid credentials open the correct role area; invalid credentials do not.
Discover	Browse categories or search for a course	Results, empty state, and request failure are visibly distinct.
Enrol	Select a free course or complete its payment flow	One learner-course enrolment exists and appears in My Learning.
Study	Open ordered content and remark a lesson complete	Progress changes once and remains stable under a retry.
Complete quiz	Submit multiple-choice or fill-blank answers	Result is returned and protected rewards are awarded only when eligible.
Track motivation	Open statistics, quests, badges, or leaderboard	Current backend state is displayed with a recoverable error state.
Ask course	Request a summary or ask a course question	Response uses course scope and provides lesson sources or fallback guidance.
Administer	Create, update, or remove learning content	A learner is rejected; an administrator can perform the valid operation.
Operate	Start services and inspect health and metrics	Required services become healthy and monitoring targets report up.

These conditions connect product language to evidence. A feature is not considered complete merely because a screen or endpoint exists; its success, empty, denied, duplicate, and failure behavior must also be understandable.

3.4 Non-functional requirements

Table 3.3: Non-functional requirements

Quality	Requirement
Usability	Screens communicate loading, empty results, validation, and recoverable failures.
Security	Passwords are hashed; signed tokens, roles, and ownership are validated.
Consistency	Retries and concurrent calls do not duplicate protected rewards.
Performance	Expensive or frequent reads use bounded caches; embeddings run asynchronously.
Maintainability	Schema changes are versioned and application concerns are separated.
Testability	Critical rules run against the real database, including rollback and concurrency.
Observability	API, database, cache, broker, and learning failures can be distinguished.
Portability	Docker reproduces infrastructure and Android builds without private Firebase files.

3.5 Requirement prioritization

Account security, course access, progress integrity, and build reproducibility are mandatory because other features depend on them. Caching, message processing, and monitoring improve performance and operations once the authoritative learning path is correct. Optional external integrations add value but must not prevent a clean developer build.

This priority influenced implementation order. PostgreSQL transactions and ownership checks guard the core path. Redis and RabbitMQ surround that path without becoming the authority for progress. Gemini, Firebase, and Supabase are configured at the edge and can be absent in local or CI environments.

Chapter 4

Technologies and Tools

4.1 Native Android

The client uses Java 17, XML layouts, minimum SDK 24, and target SDK 35. Material Components, AppCompatActivity, and ConstraintLayout provide consistent UI behavior. Navigation Component represents destinations and arguments in XML. View Binding gives compile-time access to views.

Retrofit maps REST endpoints to Java interfaces, Gson maps JSON models, and OkHttp handles transport and debug logging [15]. Glide loads images. WorkManager schedules persistent local reminders [6]. Firebase Messaging is optional and activated only by an untracked configuration file.

4.2 Spring Boot

Spring Boot 3.4 provides dependency injection, web routing, validation, security, data access, health endpoints, and external configuration [3]. Controllers translate HTTP, services contain business rules, and repositories isolate persistence. Springdoc provides OpenAPI and Swagger UI.

4.3 Persistence

PostgreSQL 16 supplies transactions, constraints, row locking, indexing, and vector extension support [11]. Flyway applies 27 ordered migrations [13]. Hibernate validates mappings without creating the schema. jOOQ supports operations where locking and conditional SQL must be explicit [4]. pgvector stores 384-dimensional MiniLM vectors beside relational course data [9].

4.4 Caching, messaging, and delivery

Redis stores caches and revocable refresh-token records [14]. RabbitMQ and Spring AMQP move content embedding away from request processing [2]. Docker Compose describes the runtime environment. Gradle and Maven wrappers provide repeatable builds. JUnit and Testcontainers exercise PostgreSQL/pgvector integration [1]. GitHub Actions coordinates backend, monitoring, and Android verification.

4.5 Technology selection rationale

Table 4.1: Technology choices and project trade-offs

Choice	Reason in EduFlex	Trade-off
Native Java/XML	Fits project convention, Android SDK access, and explicit layouts	More manual screen-state coordination than some newer UI stacks.
Spring Boot	Integrated web, validation, security, data, health, and configuration	Framework conventions require clear boundaries to avoid oversized services.
PostgreSQL	Transactions, locks, constraints, indexing, and pgvector in one system	Scaling writes still requires careful query and pool design.
jOOQ plus JPA	JPA handles mappings; jOOQ expresses locks and conditional SQL clearly	Two data-access styles need consistent repository conventions.
Redis	Fast bounded caches and refresh-token revocation	Availability and invalidation become operational concerns.
RabbitMQ	Decouples content writes from embedding work	Delivery durability needs an outbox and dead-letter policy.
Local MiniLM	Embeddings work without sending course text to a generation provider	Model changes require vector migration and reprocessing.
Docker Compose	Repeatable multi-service development and monitoring	It is a single-host development topology, not an orchestration platform.
Metrics stack	Versioned Prometheus rules and Grafana dashboards	Logs and traces remain separate future work.

The selected technologies serve specific responsibilities. PostgreSQL remains the

authority, Redis accelerates derived reads, and RabbitMQ transports deferred work. This distinction is central to failure handling: losing a cache entry should cost time, while losing an authoritative progress row would cost correctness.

Chapter 5

System Architecture and Design

5.1 Overview

The Android app sends JSON over HTTP and attaches an access token to authenticated calls. The backend owns authorization, domain rules, storage, caching, messaging, AI orchestration, and metrics.

The architecture is described through several complementary views. The context view in Figure 5.1 identifies users and systems outside EduFlex. The runtime view in Figure 5.2 then shows where requests and data travel. An optional provider can therefore fail without changing which component owns authoritative learning state.

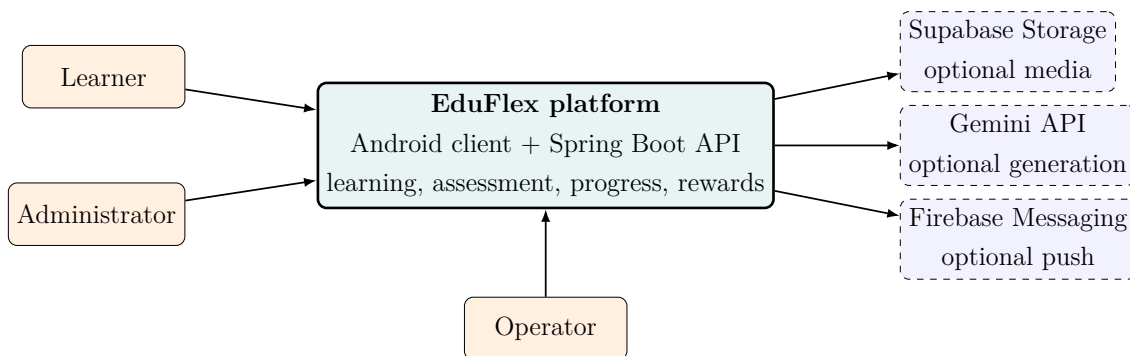


Figure 5.1: System context and external actors

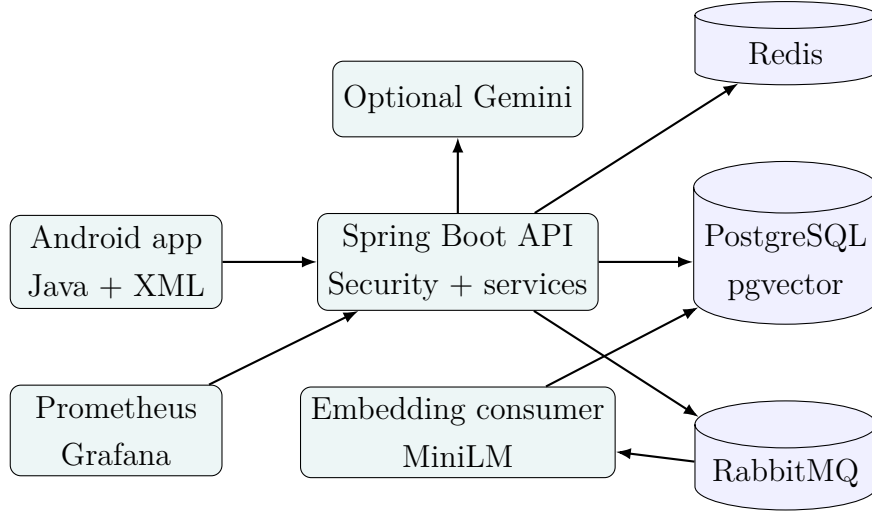


Figure 5.2: EduFlex runtime architecture

5.2 Backend structure

Controllers define transport contracts and delegate to services. DTOs isolate wire payloads from database records. Use-case classes perform validation and coordinate repositories. Repositories contain JPA and jOOQ operations. Configuration supplies security, caches, messaging, clocks, and metrics. This structure keeps business invariants out of the mobile client and HTTP layer.

Figure 5.3 makes the dependency direction explicit. Controllers call application services, and services use repositories or integration clients. Persistence code does not select HTTP responses, while Android does not calculate authoritative rewards. These boundaries allow direct service testing and reduce the effect of transport or database implementation changes.

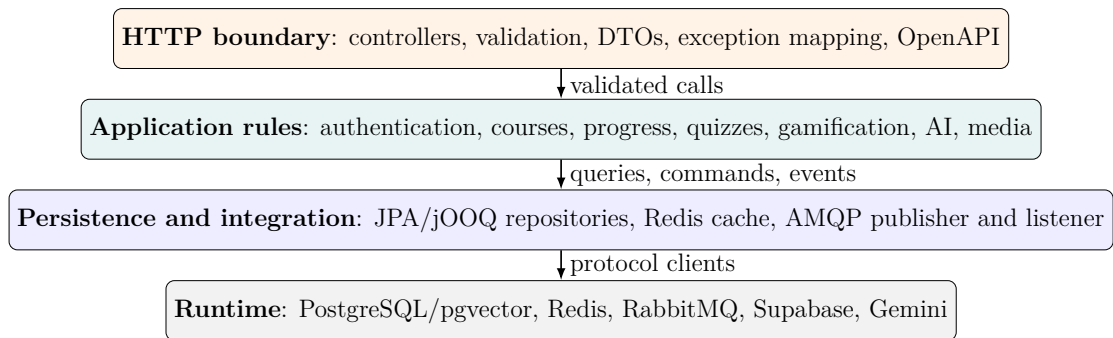


Figure 5.3: Backend responsibility layers

5.3 Android structure

Authentication activities lead to learner or administrator navigation. Learner destinations include Home, Courses, Search, Cart, Profile, Payment, Course Detail, Lesson Study, two quiz modes, Leaderboard, Reviews, Certificate, and AI Summary. Administrator destinations include Users, Courses, Lessons, and Profile. API interfaces, models, session managers, and list adapters support these screens.

Figure 5.4 shows the learner path and the separate administrator branch. Course Detail is a central decision screen that connects discovery to enrolment, study, reviews, and AI support. Stable navigation IDs and explicit arguments keep the course and lesson identity consistent across screens.

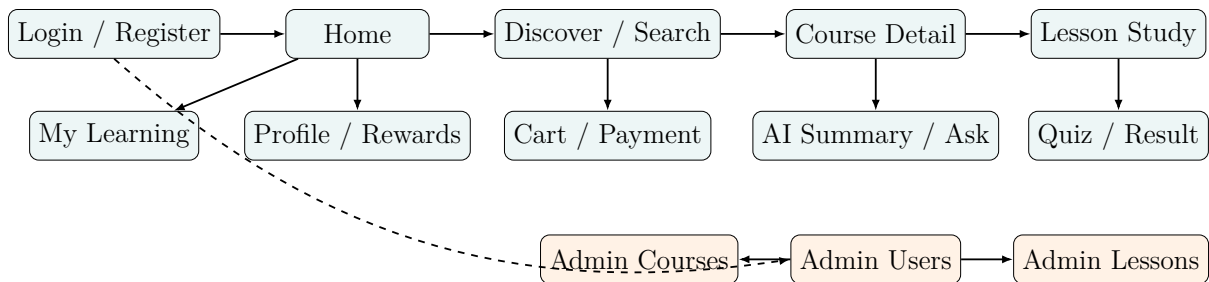


Figure 5.4: Android navigation and principal user journeys

5.4 Deployment

The base Compose stack runs the API, PostgreSQL/pgvector, Redis, and RabbitMQ. Named volumes preserve database and cache state through ordinary restarts. Health checks order startup by dependency readiness. The monitoring overlay adds Prometheus, Grafana, Alertmanager, Blackbox Exporter, PostgreSQL Exporter, and Redis Exporter.

Only selected interfaces are published to the host. PostgreSQL, Redis, exporters, and the Actuator management listener remain on the Compose network. This reduces accidental exposure while retaining local access to the API, RabbitMQ management, and loopback monitoring tools. Figure 5.5 shows the topology.

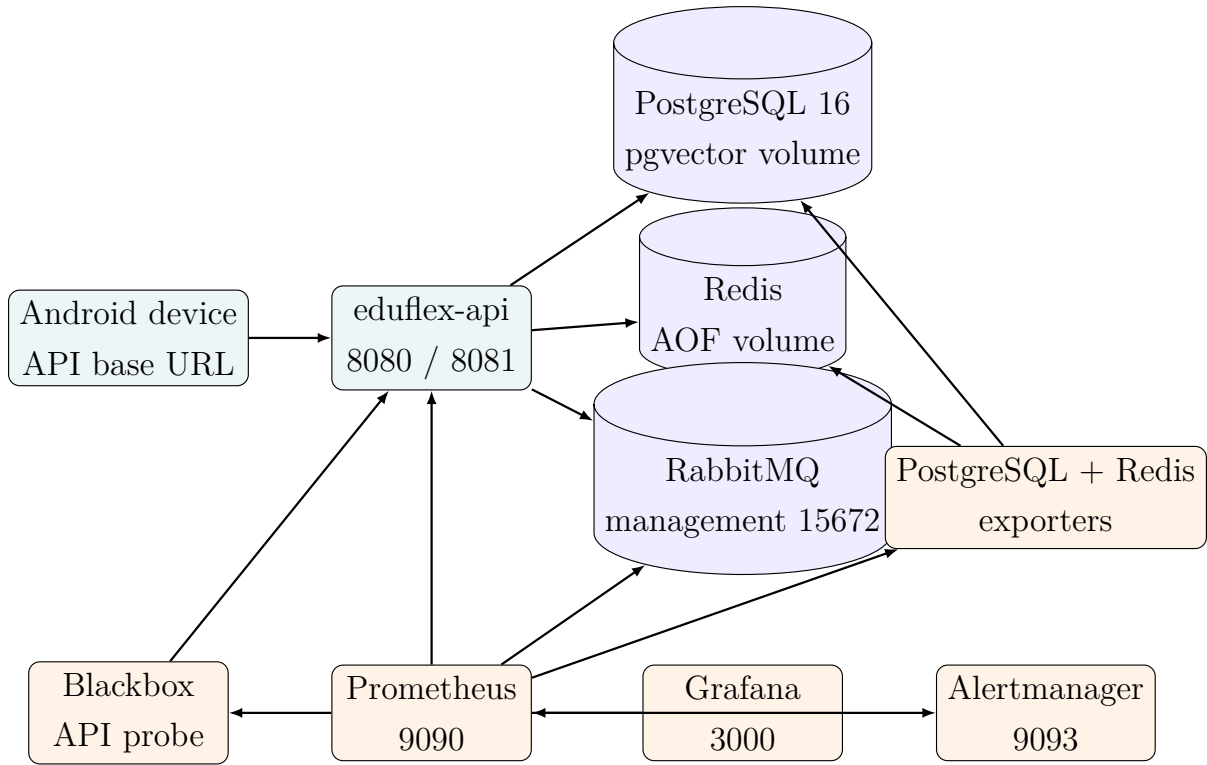


Figure 5.5: Docker Compose deployment and monitoring topology

5.5 Request and failure boundaries

A mobile operation crosses the device-to-API boundary, the security-to-application boundary, and usually an application-to-dependency boundary. Each can fail in a different way. Connectivity failures produce a retryable mobile state. Authentication and authorization failures return 401 or 403. Invalid input returns a validation response. Transaction failures roll back related writes before an error leaves the server.

This classification prevents an empty result from being confused with an error. An empty catalogue is a successful query with no matching courses. A timeout, rejected token, or unavailable database needs a distinct user message and operational signal.

Chapter 6

Database Design

6.1 Core model

Table 6.1: Principal entity groups

Group	Stored information
Users	Identity, BCrypt hash, profile, avatar, and role.
Courses and lessons	Metadata, price, images, lesson hierarchy, content, and media URL.
Quiz content	Questions, answer options, correct answers, and attempts.
Learning state	Enrolments, lesson completion, percentage, and timestamps.
Commerce and reviews	Payment records, ratings, text, and user-course relationships.
Gamification	XP, level, streaks, badges, badge awards, quests, and quest progress.
Semantic search	Course and lesson embeddings stored as pgvector columns.

Figure 6.1 focuses on business relationships rather than listing every column. Users and courses meet through enrolments, transactions, and reviews. Courses contain lessons; lessons connect to progress, questions, and attempts. Gamification is linked to the learner while badge definitions remain reusable across learners.

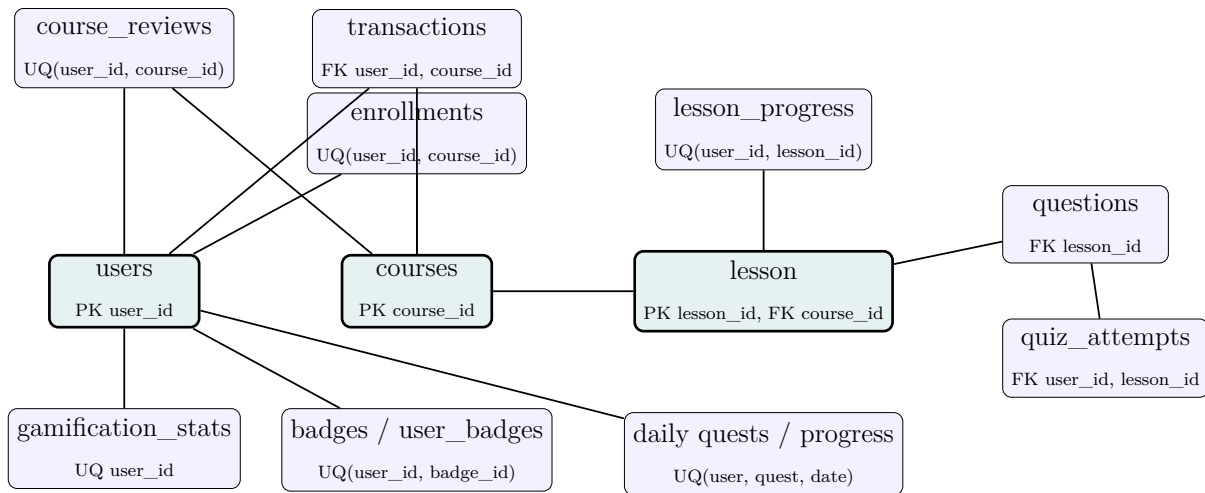


Figure 6.1: Logical database relationship model

6.2 Ownership and lifecycle

The user row is the root for authentication and learner-specific records. Removing a user cascades through progress and quiz activity where migrations explicitly define that behavior. A course owns its lesson hierarchy, and a lesson may reference a parent lesson for grouped content. Questions belong to lessons and options belong to questions. This ordering lets content deletion remove dependent assessment data in a predictable direction.

Enrolment is the bridge between catalogue data and learner activity. The backend checks it before accepting learning mutations. Progress records answer whether a specific lesson transition happened; the enrolment holds the broader course state. Gamification statistics aggregate learner rewards, while attempts retain assessment history. Keeping these concepts separate allows an attempt to be audited without recalculating an already awarded first-pass reward.

6.3 Schema evolution

The 27 migrations begin with users and progressively add courses, lessons, quizzes, enrolments, gamification, payment, reviews, media, lesson hierarchy, embeddings, quests, indexes, and constraint alignment. A clean database and an upgraded database therefore reach the same versioned schema. Hibernate uses schema validation so drift fails at startup. jOOQ source generation is explicit, allowing ordinary builds to run without a developer database.

6.4 Integrity and connections

Foreign keys connect activity to users and content. Unique constraints protect singular relationships such as enrolments and badge awards. Indexes support common course, progress, ranking, and lookup paths. Constraints remain necessary even when application code checks for an existing row because simultaneous transactions can both pass a pre-insert check.

Table 6.2: Selected integrity rules and their purpose

Data	Database rule	Invariant protected
Enrolment	Unique user–course pair	A learner joins a course once.
Lesson progress	Unique user–lesson pair	One authoritative completion transition exists.
Gamification statistics	Unique user identifier	Concurrent initialization cannot create two totals.
Badge award	Unique user–badge pair	A badge definition may be reused but its award is singular per learner.
Course review	Unique user–course pair	A learner maintains one review for a course.
Daily quest progress	Unique user–quest–date tuple	Daily work cannot create duplicate counters.
Badge definition	Unique condition type	Concurrent course completion reuses one definition.
Dependent learning rows	Foreign keys with selected cascades	User and content deletion has a defined direction.

HikariCP defaults to a maximum pool of 10 and two idle connections. Open Session in View is disabled, preventing lazy database work during response rendering. Pool active, maximum, and pending counts are exported as metrics.

6.5 Relational and vector data together

Embedding columns live beside course and lesson records, so semantic retrieval can apply normal relational filters before similarity ordering. A course question first restricts candidate material to the requested course and only then ranks vector distance. The benefit is one transaction and backup boundary for content and vectors. The trade-off

is that embedding dimensions and extension availability become part of the PostgreSQL schema and migration process.

The current MiniLM model produces 384 values. A future model change with a different dimension would require a planned migration and complete re-embedding. Recording the model identity with future vectors would make mixed-model transitions safer.

Chapter 7

Mobile Application Implementation

7.1 User experience

The UI shares a Material 3 theme, colors, typography, cards, and bottom navigation. Home presents learner status and continuation. Discover and Search expose the catalogue. My Learning shows enrolments. Course Detail links to study, reviews, and AI assistance. Profile and gamification screens present personal progress.

Data-driven screens explicitly show loading, empty results, and retryable failures. This prevents a blank screen from being interpreted as success or a crash.

The learner journey in Figure 7.1 connects UI decisions to backend state changes. Free and paid courses converge on one enrolment concept. Content and quiz lessons also converge before reward evaluation, which keeps the mobile interface focused on interaction while the backend determines correctness.

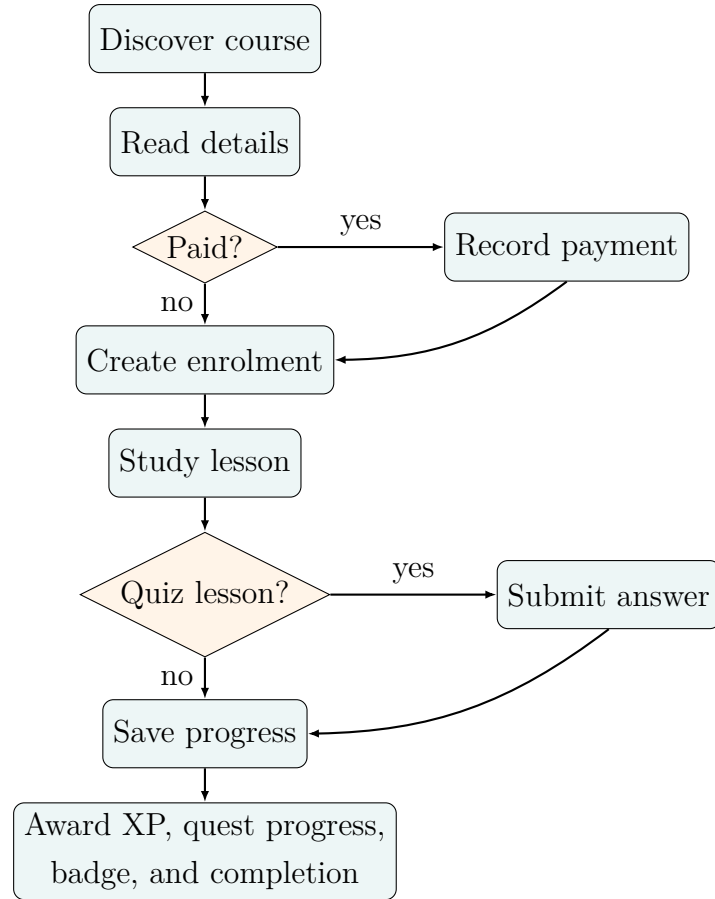


Figure 7.1: End-to-end learner course journey

After reward evaluation, an incomplete course returns the learner to the next available lesson; completed courses retain their terminal completion state.

Table 7.1: Implemented learner screens and required visible states

Screen	Successful content	Non-success state
Home	Statistics and continuation cards	Loading, absent progress, retryable request failure
Discover/Search	Course cards and filtered results	Loading, empty result, clear query, retry
My Learning	Enrolled course list	Empty route back to discovery, retry
Course Detail	Metadata, enrol/study, reviews, AI entry	Loading, missing course, network failure
Lesson/Quiz	Content, question, answer feedback	Validation, submission failure, preserved context
Profile/Rewards	Profile, XP, quests, badges	Loading, empty rewards, retry

The table is a source-level inventory, not a usability result. The recorded emulator

journey observed authentication, Home, Learn, and successful authenticated calls, but it did not capture submission-quality screenshots or cover every state. A final report submission should include annotated device captures only after repeating the journey with non-sensitive demonstration data.

7.2 Networking and authentication

The API URL is injected into `BuildConfig`; the emulator default is <http://10.0.2.2:8080/>. Retrofit interfaces map each backend domain. Debug builds enable HTTP logging and release builds disable it. After login, session management stores the access token, refresh token, role, and profile information. Provider secrets are never placed in Android build configuration.

The token manager gives networking code one location for bearer credentials. API interfaces are divided by domain so course browsing does not depend on administrator or payment details. Request and response models mirror the wire contract. This design is intentionally pragmatic for the project scale: fragments coordinate their screen state and adapters render collections, while authoritative rules remain on the server.

When a request fails, the screen retains enough context to retry without losing the selected course. Authentication failures lead back to a valid session path; ordinary network failures keep a retry action close to the failed content. Debug HTTP logging supports development diagnosis, and disabling it in release avoids routine token and payload exposure in release logs.

7.3 Learning and engagement

Lessons render content or open multiple-choice and fill-in-the-blank quiz flows. Quiz results are sent to the backend so correctness, first-pass rewards, quests, and completion are decided centrally. Gamification displays XP, levels, streaks, quests, badges, and leaderboard position. `WorkManager` provides local reminders; optional Firebase Messaging provides remote notifications.

Course completion is derived from required content and assessment state rather than from a button controlled by the device. The client submits the event and renders the returned result. This allows another device or a retried request to observe the same server state. The progress state model is discussed in the reliability chapter.

7.4 UI quality and accessibility considerations

Material components provide consistent touch targets, typography, colors, and control behavior. Lists use dedicated empty views rather than leaving an empty `RecyclerView`.

Search supports keyboard submission, clearing, and retry. My Learning offers a route back to discovery when the learner has no enrolments.

The acceptance checklist includes normal and enlarged fonts, keyboard visibility, back navigation, loading, empty, and offline states. Further accessibility work should add automated contrast and content-description checks plus TalkBack traversal tests. Those checks are proposed work and are not claimed as completed evaluation.

7.5 Build variants

The debug APK is installable for development. The release variant enables minification, optimization, and resource shrinking. CI verifies release APK and AAB generation, but the artifacts remain unsigned and are not store-ready.

Chapter 8

Backend and Security Implementation

8.1 REST API

Routes cover users, authentication, courses, lessons, enrolment, progress, quizzes, payments, reviews, media, gamification, badges, quests, and administration. Swagger UI is available at `/swagger-ui/index.html` and OpenAPI at `/v3/api-docs`. Validation and centralized exception handling make input and failure responses consistent.

The API is organized around resources rather than individual screens. Android may change its navigation without renaming the progress or course contract. Read routes return DTOs shaped for clients; commands accept JSON, form data, or multipart content according to the operation. Figure 8.1 groups routes by the security decision that protects them.

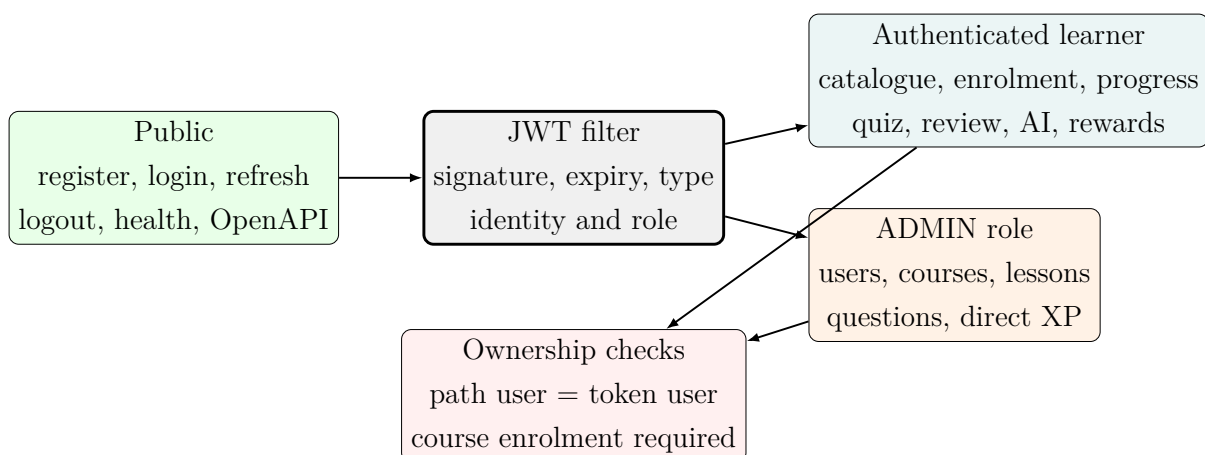


Figure 8.1: API groups and authorization boundaries

8.2 API contract groups

Account contracts create sessions and maintain profiles. Catalogue contracts return courses, search results, owned courses, reviews, summaries, and grounded answers. Learning contracts cover enrolment, lesson hierarchy, progress, and the two quiz submission forms. Gamification contracts return statistics, check-in results, quests, badges, and leaderboard entries. Media uses multipart requests because binary content has different validation and size requirements from JSON.

Administrative commands are collected under `/api/admin` for list, create, update, and delete workflows, with additional privileged mutations explicitly covered by security rules. The complete generated contract remains available through OpenAPI; the report groups related endpoints so the reader can understand responsibility and access without reproducing a large machine-generated document.

Table 8.1: Representative API contracts

Operation	Contract, success, and failure boundary
Login	POST <code>/api/user/login</code> accepts email and password. Success returns access and refresh tokens, user, and role; validation or invalid credentials fail.
Refresh	POST <code>/api/auth/refresh</code> accepts a refresh token. Success returns a new access token; an unknown user or malformed token returns 400, and an expired or revoked token returns 401.
Enrol	POST <code>/api/enrollment/{courseId}/register</code> accepts the user ID. Success creates the singular learner-course enrolment; unauthenticated and wrong-owner calls are rejected.
Complete lesson	POST <code>/api/progress/lesson</code> accepts form fields <code>lessonId</code> and <code>userId</code> . Success returns the progress percentage after a protected transition; invalid ownership, enrolment, or lesson state fails.
Submit quiz	POST <code>/api/quiz/submit-multiple-choice</code> accepts user, lesson, and answers. Success returns score, course progress, XP, and completed quests; a response-loss retry may add an attempt.
Ask course	POST <code>/api/course/{courseId}/ask</code> accepts a question of at most 500 characters. Success returns an answer, lesson sources, and provider flag; provider failure uses the fallback.

The progress command remains form-encoded for compatibility with the Android client. A representative successful response is:

Listing 8.1: Lesson-completion response contract

```
1 {  
2   "success": true,  
3   "message": "Lesson progress saved",  
4   "newProgressPercent": 50.0  
5 }
```

The request has no client idempotency key. Protected idempotency comes from the unique user-lesson row and conditional transaction. Quiz attempts have a different boundary and can still duplicate history after a lost response, as discussed later.

8.3 Authentication and authorization

Passwords use BCrypt. Login returns a 15-minute access token with identity, email, role, and token type, plus a 30-day refresh token with a unique identifier. Refresh state is held in Redis and logout revokes it. Spring Security uses stateless sessions. Public routes include registration, login, recovery, refresh, logout, health, and API documentation. Other routes require authentication. Administrator operations require the **ADMIN** role; learning mutations also validate ownership.

Figure 8.2 follows both the initial login and later refresh. The access token is self-contained and short-lived. The refresh token is longer-lived, but its identifier is checked in Redis, which gives logout a server-side revocation point. A refresh token is not accepted as an access token because the signed token also carries a token-type claim.

Logout deletes the selected refresh-token record. It prevents that refresh token from minting another access token; it does not immediately invalidate an already issued 15-minute access token. This bounded residual session is a consequence of stateless access-token validation.

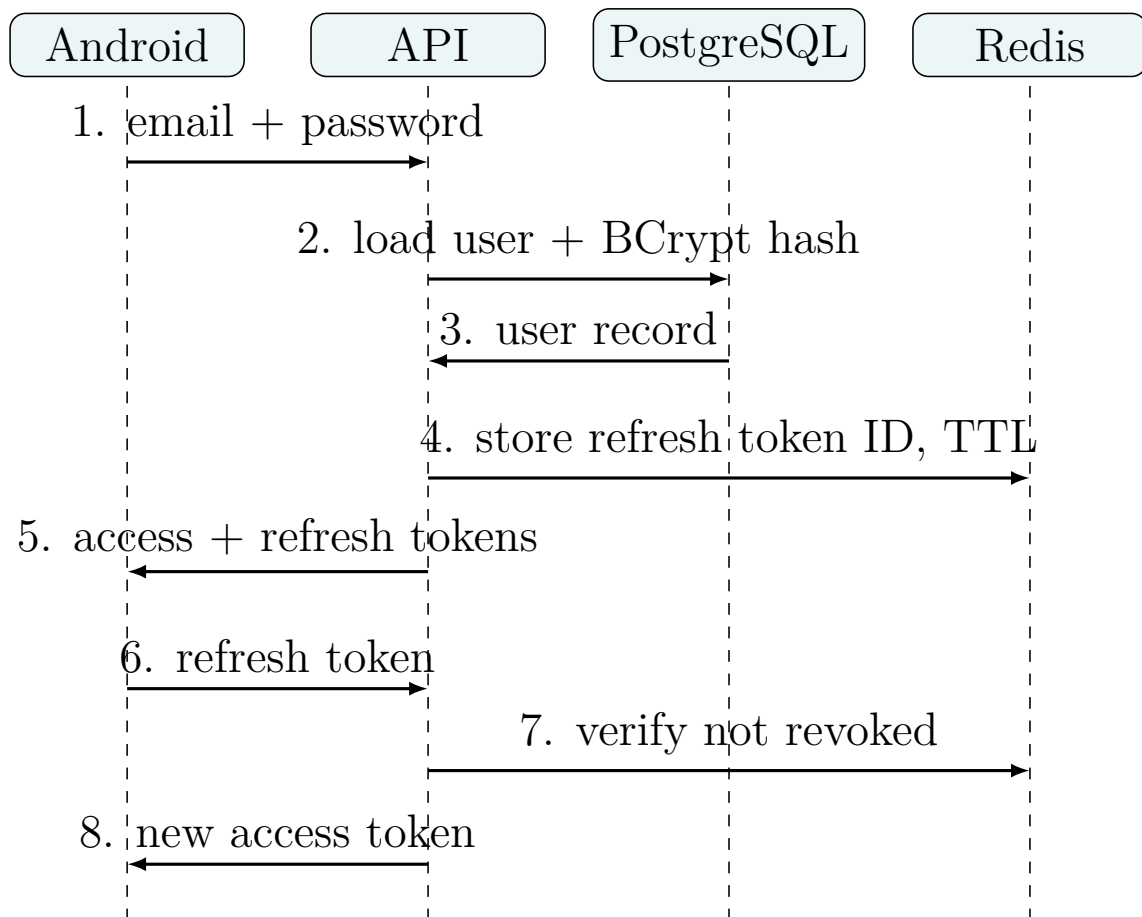


Figure 8.2: Login and access-token refresh sequence

Authorization has two levels. Route rules decide whether a role may reach an endpoint. Application rules then check whether the authenticated learner owns the requested user path and is enrolled in the relevant course. Role checks alone cannot protect a request such as “save progress for user B” made by authenticated user A.

8.4 Media

The server can upload avatars, course images, and lesson videos to Supabase Storage when configured. The secret stays on the backend. The core service can run without these optional credentials.

8.5 Redis cache policy

Table 8.2: Configured Redis cache lifetimes

Courses	5 min	Lessons	5 min
Quizzes	10 min	Leaderboard	1 min
User statistics	2 min	Badge catalogue	30 min
User badges	5 min	Reviews	3 min
Course catalogue	5 min	AI summaries	6 hours
Semantic search	15 min		

Writes evict relevant entries. Embedding processing clears semantic and summary results affected by changed content. Cache statistics support measurement.

Figure 8.3 shows the cache-aside behavior. A cache hit avoids a database query. A miss loads authoritative data and stores a bounded copy. Mutations evict domain caches rather than attempting to modify several serialized cached representations in place.

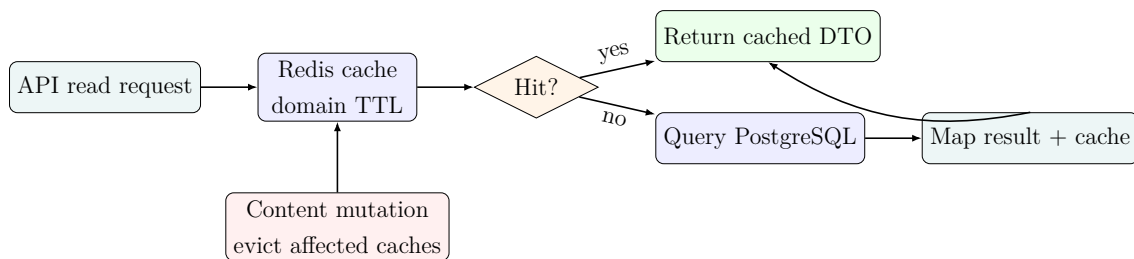


Figure 8.3: Cache-aside read and invalidation flow

Short TTLs are used for rapidly changing learner state, while the badge catalogue and generated summaries can live longer. This is a consistency trade-off, not a guarantee that data remains unchanged for the whole TTL. Explicit eviction narrows the stale interval after known writes; expiration protects against missed eviction and limits memory growth.

Chapter 9

Reliability, Concurrency, and Messaging

9.1 Transactional updates

One learning action may update progress, XP, quests, enrolment completion, and badges. These operations run inside one transaction. If an injected failure occurs after intermediate writes, all changes roll back and a later retry completes once.

9.2 Per-learner serialization

The service locks the learner's statistics row with PostgreSQL `SELECT FOR UPDATE`. Requests for one learner serialize at the critical transition, while different learners proceed independently. Conditional updates decide whether a transition is new and uniqueness constraints provide a final safeguard. Protected transitions include lesson completion, daily check-in, enrolment, first quiz pass, course completion, quest completion, and badge award.

Figure 9.1 shows the lifecycle seen by repeated requests. A duplicate does not reverse the learner or create a second protected reward; it returns the state that already won the race. This property matters on mobile networks, where the server may commit even when the client never receives the response.

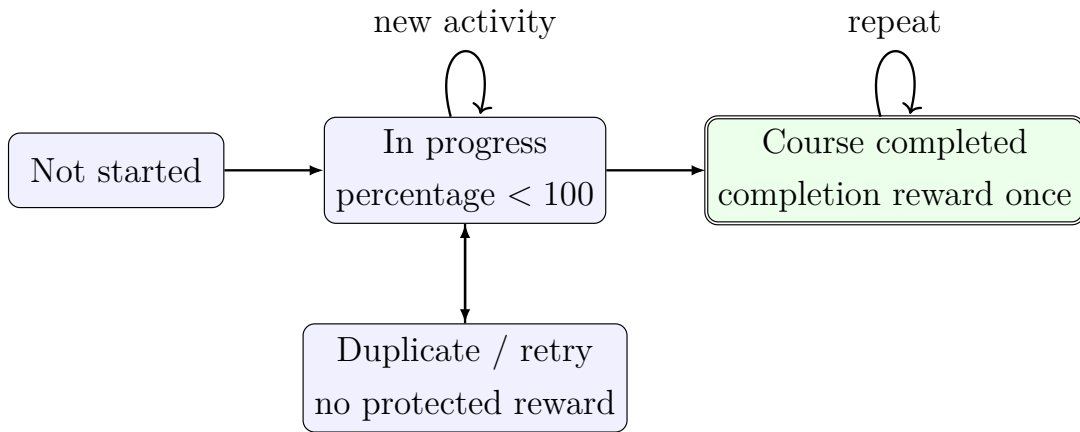


Figure 9.1: Learning-progress state and retry behavior

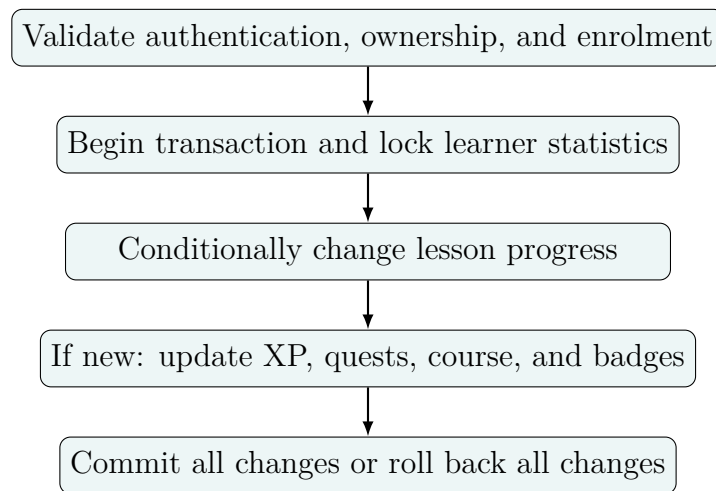


Figure 9.2: Concurrency-safe progress transaction

9.3 RabbitMQ embeddings

Course and lesson services publish a content-change event after transaction commit. A durable queue feeds a listener with prefetch 10 and three retry attempts. The listener creates a MiniLM vector, stores it, and invalidates related caches. Publication outcomes and processing duration are measured.

Figure 9.3 explains the ordering. Publishing before commit could let the consumer read old content or embed a write that later rolls back. After-commit publication avoids that ordering error and keeps model inference outside the HTTP response time.

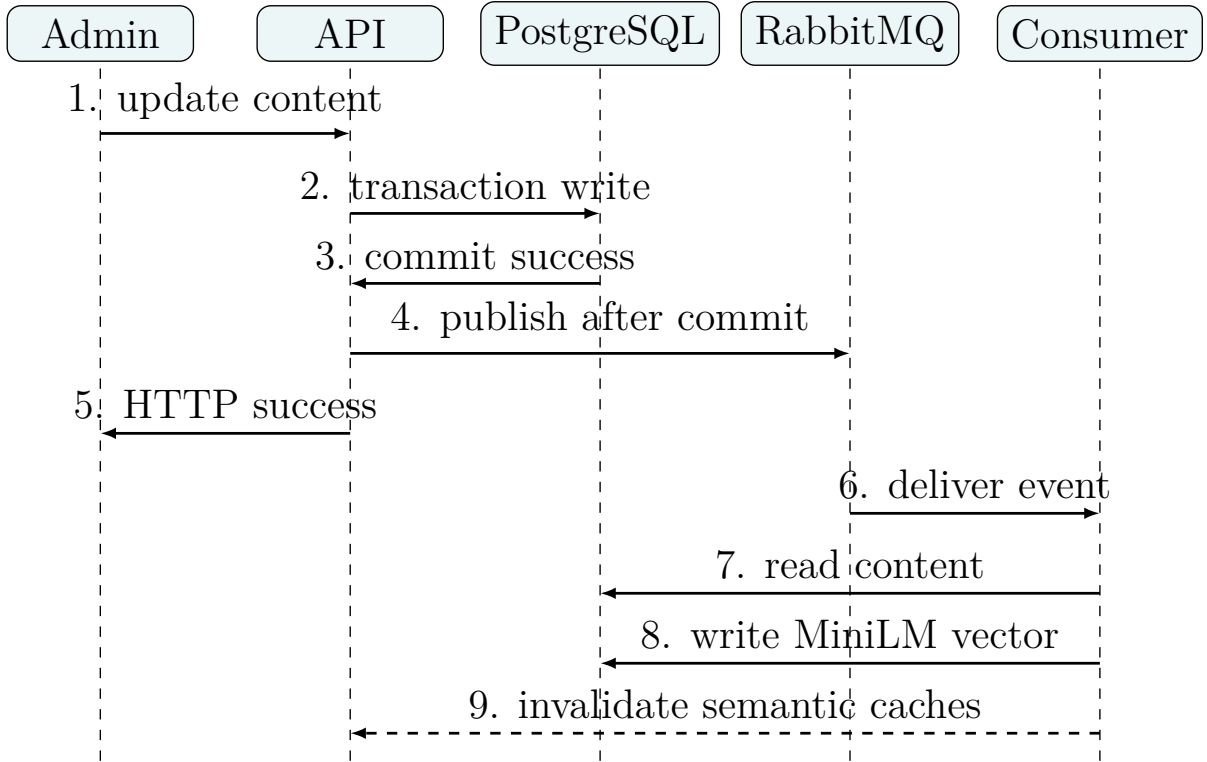


Figure 9.3: Post-commit content embedding sequence

The design currently has no transactional outbox or publisher confirmation. A committed content write can therefore exist without a durable event if publication fails. Operators must react to the alert and replay affected content.

9.4 Known retry boundary

Quiz submissions create attempt records and do not accept a client idempotency key. A response-loss retry can create another attempt and increment an unfinished quest. First-pass XP, course XP, quest-completion XP, progress, and badges remain protected.

9.5 Failure analysis

Table 9.1: Reliability mechanisms and remaining boundaries

Failure	Current protection	Remaining action
Duplicate lesson request	Conditional transition and unique model	Return existing state without new XP.
Concurrent reward writes	Per-learner row lock and transaction	Observe lock time and pool pressure.

Failure	Current protection	Remaining action
Exception after partial work	Transaction rollback	Retry the complete operation.
RabbitMQ publish failure	Content stays committed; failure metric increases	Replay content and add an outbox.
Consumer failure	Listener retries three times	Add explicit dead-letter handling.
Lost quiz response	Protected rewards remain single	Add an idempotency key for attempts.
Redis unavailable	Readiness and monitoring expose dependency failure	Restore Redis and define degraded behavior.

Correctness and throughput have different meanings. Serializing one learner’s reward mutations is intentional because those operations contend for one logical aggregate. Different learners lock different rows and can proceed independently. A load test is still required to measure connection-pool, database CPU, and lock-duration limits.

Chapter 10

AI-Assisted Learning

10.1 Retrieval

The local all-MiniLM-L6-v2 model produces 384-dimensional course and lesson vectors. A learner question is embedded and compared with chunks from the selected course using pgvector. Constraining retrieval by course reduces unrelated context.

Embedding is a representation step rather than an answer-generation step. Vector distance lets differently worded questions locate related lessons. The relational course filter still defines the permitted knowledge boundary: SQL supplies ownership and scope, while vector similarity supplies relevance.

10.2 Grounded generation and fallback

Retrieved chunks become context for optional Gemini generation. The result contains source information so the UI can identify supporting material. Expensive summaries and search results are cached. Without `GEMINI_API_KEY`, summaries use a deterministic overview and questions return retrieved lesson suggestions, allowing local development and tests without provider access.

The backend returns sources with the answer instead of presenting generated prose alone. This gives the learner a route back to the lesson and lets an evaluator inspect retrieval quality. If retrieval selects weak evidence, fluent wording does not make the evidence stronger; source visibility makes the weakness observable.

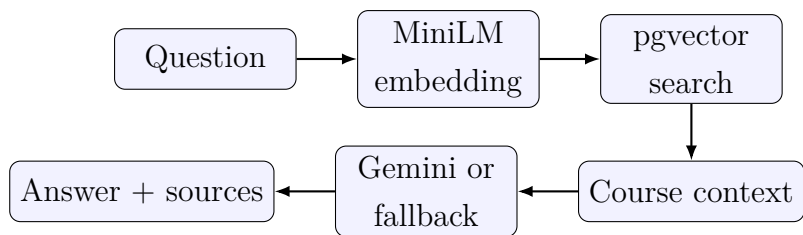


Figure 10.1: Grounded course-question flow

10.3 Responsible boundaries

Prompts, learner IDs, emails, tokens, and questions are excluded from metric labels. Answers remain attributable to stored course content and are not treated as authoritative academic assessment. Provider outcomes use bounded metric labels.

10.4 Quality evaluation approach

The current automated test proves deterministic fallback behavior. A broader AI evaluation should prepare representative questions for each course, expected lesson sources, and educator-reviewed answer criteria. Retrieval can then be measured through source recall and ranking, while generated responses can be judged for groundedness, completeness, and unsupported claims. Latency and provider failure rate should be reported separately from content quality.

This evaluation remains future work because the repository does not contain a labelled educational benchmark. The report describes the implemented retrieval pipeline without claiming a measured pedagogical quality threshold.

Chapter 11

DevOps, Deployment, and Monitoring

11.1 Containerization and configuration

Docker Compose builds the API and runs PostgreSQL/pgvector, Redis with append-only persistence, and RabbitMQ. Health checks guard dependency readiness. Example environment files document variable names; real credentials stay untracked.

11.2 Continuous integration

GitHub Actions runs three dependent jobs. The backend job uses JDK 21 and executes Maven/Testcontainers tests. The monitoring job starts an isolated stack and validates it. The Android job installs SDK 35, runs lint, builds debug and release APKs, and builds a release AAB. Reports and artifacts are retained for 14 days.



Figure 11.1: Continuous integration order

11.3 Metrics, dashboards, and alerts

Actuator and Micrometer expose JVM, HTTP, HikariCP, and custom metrics on an internal management listener. Prometheus scrapes the API, its main-listener probe, PostgreSQL, Redis, and RabbitMQ exporters [12]. Custom metrics cover progress, rewards, statistics-lock duration, embedding publication and processing, and AI outcomes.

Grafana provisions Service Overview, Dependencies, and Learning Operations dashboards [7]. Alerts cover API availability, absent targets, 5xx rate, p95 latency, pool

contention, queue backlog, absent consumers, publication failure, RabbitMQ alarms, Redis evictions, and PostgreSQL deadlocks. Alertmanager groups alerts locally.

Figure 11.2 separates telemetry sources from storage, display, and local alert inspection. A healthy Actuator scrape proves that the management listener is alive; the Blackbox probe separately tests the client-facing API port. The two signals prevent a healthy metrics port from hiding a failed application listener.

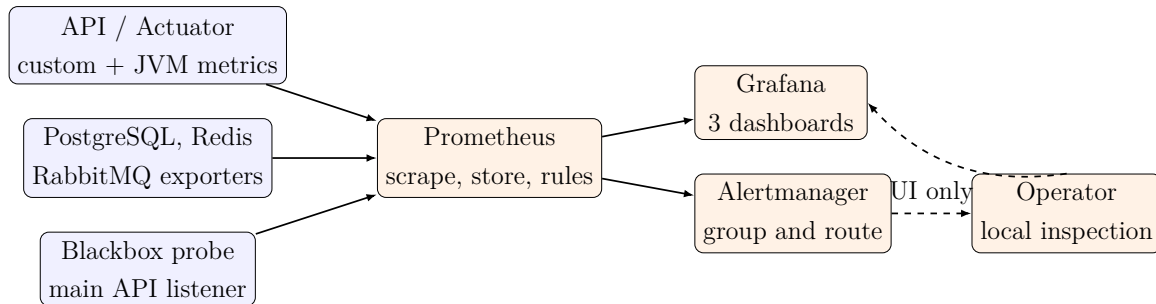


Figure 11.2: Metrics, visualization, and alerting flow

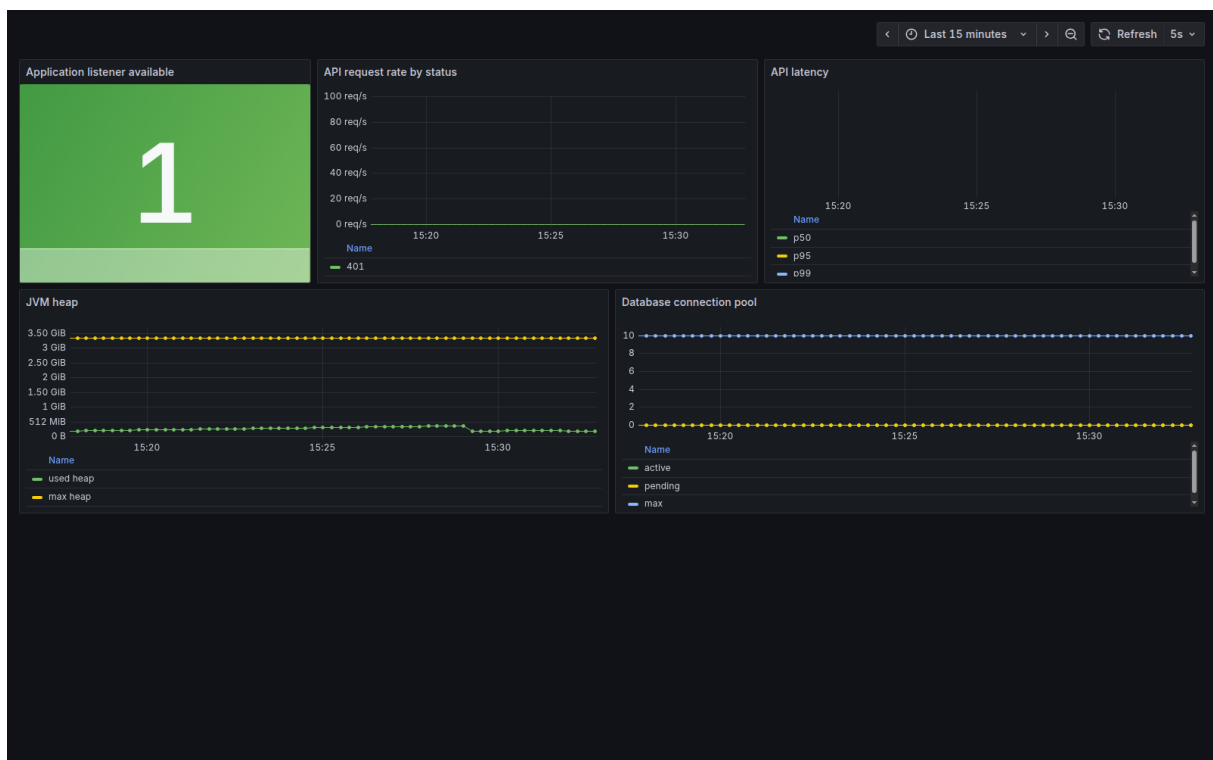


Figure 11.3: Provisioned Grafana Service Overview running against the local stack

Figure 11.3 is a real local capture rather than a mock-up. It shows the independent application-listener probe, request status, latency panels, JVM heap, and Hikari connection-pool capacity. The near-idle request and latency panels reflect the capture workload; they must not be interpreted as a benchmark.

11.4 Operational security

Liveness and readiness are exposed on the main API port. The monitoring profile exposes only health and Prometheus on internal port 8081. Grafana, Prometheus, and Alertmanager bind to loopback locally. The PostgreSQL exporter uses a dedicated `eduflex_monitor` role with `pg_monitor`. Prometheus retention is bounded to seven days or 2 GB.

The current Alertmanager receiver groups alerts for inspection in its local web UI but does not send email, chat, or paging notifications. External delivery is therefore a future deployment task rather than an implemented operational guarantee.

11.5 Operational interpretation

The dashboards correspond to questions asked during an incident. Service Overview shows whether the API is unreachable, slow, failing, or waiting for database connections. Dependencies narrows pressure to PostgreSQL, Redis, or RabbitMQ. Learning Operations shows progress transitions, rewards, row-lock time, embedding work, and AI outcomes.

An alert starts an investigation; it does not prove one cause. High latency together with Hikari pending requests points toward database work or pool saturation. A growing embedding queue with zero consumers points toward listener startup or broker connectivity. Redis eviction requires inspecting memory policy and workload before capacity changes. The monitoring runbook records these decision paths and recovery checks.

Chapter 12

Testing and Evaluation

12.1 Strategy

Verification combines unit tests, database integration tests, live API journeys, Android emulator checks, lint, builds, and monitoring smoke tests. Attention focuses on token semantics, rollback, ownership, duplicate rewards, concurrency, and startup.

Unless a row states otherwise, the recorded evidence was produced on 24–25 September 2026 from Git revision `3d21d21` on branch `dev/set-up-message-queue`. The detailed scenario record is versioned in `docs/reliable-progress-verification.md`; monitoring procedures and the dated smoke result are in `docs/monitoring-runbook.md`. Naming the revision matters: test totals are historical observations and must be rerun after functional changes.

No single test layer proves the whole system. Unit tests give fast feedback for token and fallback behavior. Testcontainers supplies real transaction, constraint, and lock semantics. The live API journey checks security filters, serialization, persistence, and messaging through HTTP. The emulator adds Android navigation and contract mapping. Monitoring verification confirms that operational signals remain usable after startup.

Table 12.1: Evidence mapped to quality goals

Goal	Evidence	What it establishes
Security	JWT tests and rejected HTTP flows	Token type, role, and ownership in tested routes.
Consistency	PostgreSQL concurrency suite	Single protected transitions under duplicate calls.
Atomicity	Injected transaction failure	Related progress and rewards roll back together.
Integration	26-check live HTTP journey	Main backend workflow across real dependencies.
Mobile compatibility	Contract test and API 35 emulator	Token mapping and representative navigation.
Delivery	Lint, APK, and AAB builds	Source and shrinking configuration compile.
Observability	Targets, dashboards, and alert drill	Metrics and alert lifecycle work locally.

Table 12.2: Claim-to-evidence traceability

Report claim	Repository evidence	Boundary
Protected rewards are single under duplicate concurrency	Progress concurrency integration test; final database assertions after 100 synchronized calls	Correctness for defined scenarios, not capacity
Related writes are atomic	Injected exception and retry integration scenario	One transaction path and PostgreSQL configuration
Authorization rejects wrong ownership	Integration scenarios plus live HTTP journey	Exercised routes, not a complete penetration test
Embedding crosses RabbitMQ	HTTP journey waits for stored vector after content change	Successful delivery path; no outbox guarantee
Refresh revocation works	JWT unit tests and refresh/logout HTTP checks	Refresh token only; issued access token remains valid until expiry
Android consumes the contract	Login response unit test and API 35 emulator journey	Representative navigation, not full device coverage

Report claim	Repository evidence	Boundary
Monitoring detects API loss	Versioned rules, seven tar-gets, and stop/recovery drill	Local Compose; receiver has no external notification route
Release source compiles with shrinking	Gradle release APK/AAB tasks	Unsigned inspection arti-facts

12.2 Backend results

The recorded run used Java 21, Docker Engine 29.8.1, and PostgreSQL 16/pgvector. All 27 migrations ran. The result was 20 tests with zero failures, errors, or skips; 18 scenarios belong to the progress concurrency integration suite.

Table 12.3: Selected verified scenarios

Scenario	Observed result
100 concurrent completions of one lesson	One progress row and exactly 20 XP.
100 concurrent daily check-ins	One date transition and exactly 10 XP.
100 concurrent first statistics reads	One row without uniqueness failure.
Repeated lesson completion	Timestamp and XP preserved.
Different lessons completed concurrently	Both rewards and percentage correct.
Concurrent first quiz passes	First-pass and course XP awarded once.
Two learners complete one course	One badge definition and one award per learner.
Injected failure after related writes	Full rollback; retry succeeds once.
Unauthorized and wrong-owner calls	Rejected without state mutation.
Daily boundary with fixed clock	Same-day, next-day, zone, and future cases pass.

These tests establish correctness for the tested cases; they are not a throughput

benchmark and do not claim production latency.

The concurrency harness starts 100 worker threads together and permits up to 32 simultaneous test database transactions. Assertions inspect final database state, not only successful responses. This matters because every call could return success while still duplicating XP internally. A pinned PostgreSQL/pgvector image and Flyway startup keep tested database behavior close to the local runtime.

12.3 End-to-end and Android results

A live HTTP journey completed 26 checks with no failures. It covered administrator and learner registration, protected content creation, catalogue reads, cross-user rejection, enrolment, progress, quiz retry, XP totals, embedding completion, refresh, logout, and revoked-token rejection.

The debug APK was exercised on an Android 15 API 35 emulator. A learner registered, logged in, opened the main activity, accepted notification permission, viewed Home, opened Learn, and received successful authenticated statistics, quest, and enrolment responses. A unit contract test protects access-token response mapping.

The journey also waited for RabbitMQ embedding completion, demonstrating that a content update can travel through broker and consumer into vector storage. It checked token refresh followed by logout and rejection of the revoked token. These steps join otherwise isolated features into a meaningful lifecycle.

12.4 Build and monitoring results

Android debug lint, debug APK, release APK, and release AAB tasks succeeded with release shrinking enabled. Monitoring verification checked Compose configuration, alert rules, metric protection, seven Prometheus jobs, and three dashboards. A controlled API stop caused the availability alert to fire after one minute and resolve after recovery.

The drill checks both directions of the alert lifecycle. Firing demonstrates detection; resolution confirms that the rule, scrape path, and restored service agree after recovery instead of leaving a stale incident.

12.5 Evaluation limits

Emulator checks do not cover every device, screen size, or accessibility setting. No formal usability study or load-test dataset has been completed. External providers require separate credentials. These limits prevent unsupported claims and define future evaluation work.

A formal next evaluation should record task completion time and error rate for new learners, accessibility findings, p50/p95/p99 latency under a defined workload, database

and queue saturation points, and recovery time after dependency failures. Each result should state dataset size, hardware, concurrency, cache state, and the script used to reproduce it.

12.6 Performance claims and missing measurements

Redis and RabbitMQ change where work occurs, but their presence alone does not prove a performance improvement. The project has not yet recorded uncached versus cached latency, synchronous versus queued embedding response time, sustained throughput, or resource saturation. Consequently, this report describes the intended performance mechanism and does not attach numerical speed-up claims to it.

Table 12.4: Required performance experiment design

Question	Controlled comparison	Reportable measures
Does caching help?	Cold/miss, warm/hit, and post-eviction reads with the same dataset	p50/p95/p99 latency, database calls, hit ratio
Does queuing shorten writes?	Content update with synchronous embedding base-line versus post-commit publication	HTTP latency, queue delay, end-to-end embedding time
Where does concurrency saturate?	Increasing learners and requests with fixed hardware and pool settings	Throughput, errors, lock time, CPU, pool pending
How does failure recover?	Stop and restore Redis, RabbitMQ, API, and consumer independently	Detection time, recovery time, lost/replayed work

Chapter 13

Conclusion and Future Work

13.1 Conclusion

EduFlex delivers a mobile learning experience supported by a secure, transactional, and observable backend. It covers the path from Material-based Android screens to REST services, relational storage, caches, messaging, semantic retrieval, integration tests, containers, and dashboards.

The central achievement is trustworthy learning state. Progress and gamification are coordinated through transactions, row locks, conditional transitions, and constraints. Tests demonstrate that concurrent duplicate calls do not duplicate protected rewards. Caching and asynchronous embedding improve the design without moving authority away from PostgreSQL.

Observability is part of the implementation: metrics represent infrastructure and learning operations, dashboards are version-controlled, and availability alerting has been exercised. CI combines backend correctness, monitoring, and Android delivery.

13.2 Future work

1. add a transactional outbox, publisher confirmation, and reconciliation;
2. add client idempotency keys and a reward ledger for quiz and payment retries;
3. add structured centralized logs and OpenTelemetry traces;
4. configure external alert delivery and encrypted backups with restore drills;
5. load-test catalogue, progress, and AI paths and define measured objectives;
6. expand Android instrumentation, accessibility, offline, and device testing;
7. implement secure release signing and an approved store workflow; and

8. evaluate AI answers with a labelled dataset and educator review.

13.3 Portfolio summary

Designed and implemented a native Android learning platform with a Spring Boot API, PostgreSQL/pgvector, Redis, RabbitMQ, JWT security, grounded AI assistance, and Prometheus with Grafana. Built concurrency-safe progress and reward transactions and verified single-reward behavior under 100 simultaneous duplicate requests with Testcontainers.

Appendix A

Build and Verification Commands

Listing A.1: Start the base environment

```
1 cp .env.example .env
2 cp backend/.env.example backend/.env
3 docker compose --env-file backend/.env \
4   -f backend/docker-compose.yml up --build -d
```

Listing A.2: Run backend tests

```
1 cd backend
2 ./mvnw --batch-mode test
```

Listing A.3: Run Android verification

```
1 ./gradlew --no-daemon \
2   :app:lintDebug :app:assembleDebug \
3   :app:assembleRelease :app:bundleRelease
```

Listing A.4: Verify monitoring

```
1 docker compose --env-file backend/.env \
2   -f backend/docker-compose.yml \
3   -f backend/docker-compose.monitoring.yml up --build -d
4 bash backend/scripts/verify-monitoring.sh
```

Appendix B

Selected API Groups

Route group	Responsibility
/api/user	Registration, login, recovery, and profile update.
/api/auth	Token refresh and logout.
/api/course	Catalogue, search, reviews, summary, and questions.
/api/lesson	Lesson creation and retrieval.
/api/enrollment	Learner enrolment and reads.
/api/progress	Lesson completion and course progress.
/api/quiz	Question retrieval and submissions.
/api/payment	Payment record processing.
/api/users/...	Statistics, streaks, quests, badges, and leaderboard.
/api/media	Avatar, image, and video upload.
/api/admin	Protected user and content administration.

Appendix C

Configuration Reference

Variable	Purpose	Scope
API_BASE_URL	Android REST endpoint, including trailing slash	Mobile
SPRING_DATASOURCE_URL	JDBC connection for direct backend execution	Backend
SPRING_DATASOURCE_USERNAME	Application database user	Backend
SPRING_DATASOURCE_PASSWORD	Application database password	Backend
JWT_SECRETKEY	HMAC signing secret for access and refresh tokens	Backend
REDIS_PASSWORD	Redis authentication	Infra
RABBITMQ_USERNAME	Broker login name	Infra
RABBITMQ_PASSWORD	Broker login password	Infra
EDUFLEX_TIME_ZONE	Calendar zone for streaks and daily rewards	Backend
GEMINI_API_KEY	Optional server-side generation credential	Optional
GEMINI_MODEL	Optional generation model selection	Optional
SUPABASE_URL	Optional media-storage project URL	Optional
SUPABASE_SECRET_KEY	Optional server-side storage credential	Optional
GRAFANA_ADMIN_PASSWORD	Local Grafana administrator password	Monitor
POSTGRES_MONITOR_PASSWORD	Dedicated exporter role password	Monitor
EDUFLEX_ENVIRONMENT	Stable environment label on metrics	Monitor

Real values belong in untracked environment files or a deployment secret store. Provider keys and passwords must not appear in screenshots, source control, metric labels, build logs, or the submitted report archive.

Appendix D

Diagram and Evidence Index

View	Repository evidence represented
System context	Android application, backend, optional Supabase, Gemini, and Firebase integrations.
Runtime architecture	Compose services, API dependency configuration, and embedding listener.
Backend layers	Controller, service, repository, security, configuration, and monitoring packages.
Android navigation	Learner and administrator navigation XML plus screen classes.
Deployment topology	Base and monitoring Compose files, exposed ports, and named volumes.
Database model	Flyway migrations, generated records, entities, and foreign keys.
API boundaries	Controller mappings and Spring Security route rules.
Authentication sequence	Login use case, JWT utility, refresh service, and Redis revocation.
Learner journey	Course, payment, enrolment, lesson, quiz, progress, and reward services.
Progress state	Conditional progress transitions and concurrency integration tests.
Cache flow	Redis configuration plus cacheable and eviction annotations.
Embedding sequence	After-commit publisher, RabbitMQ binding, listener, and embedding service.

View	Repository evidence represented
AI retrieval	MiniLM embedding service, pgvector search, Gemini client, and fallback test.
CI flow	GitHub Actions backend, monitoring, and Android jobs.
Monitoring flow	Prometheus jobs and rules, exporters, Grafana provisioning, and Alertmanager.

This index makes each visual auditable. The figures intentionally simplify class and column detail, but their relationships follow the repository rather than a hypothetical target architecture.

Appendix E

Submission Checklist

Replace title-page placeholders, use any declaration required by the institution, and insert real screenshots. Confirm page-count and formatting rules, and update verification dates after new test runs. Never include secrets, private Firebase configuration, signing files, real user data, or access tokens in the report.

Bibliography

- [1] AtomicJar. Testcontainers for java. <https://java.testcontainers.org/>, 2026. Accessed 25 September 2026.
- [2] Broadcom. Rabbitmq documentation. <https://www.rabbitmq.com/docs>, 2026. Accessed 25 September 2026.
- [3] Broadcom. Spring boot reference documentation. <https://docs.spring.io/spring-boot/index.html>, 2026. Accessed 25 September 2026.
- [4] Data Geekery GmbH. jooq user manual. <https://www.jooq.org/doc/latest/manual/>, 2026. Accessed 25 September 2026.
- [5] Google. Guide to app architecture. <https://developer.android.com/topic/architecture>, 2026. Accessed 25 September 2026.
- [6] Google. Workmanager overview. <https://developer.android.com/develop/background-work/background-tasks/persistent/getting-started>, 2026. Accessed 25 September 2026.
- [7] Grafana Labs. Grafana documentation. <https://grafana.com/docs/grafana/latest/>, 2026. Accessed 25 September 2026.
- [8] Juho Hamari, Jonna Koivisto, and Harri Sarsa. Does gamification work? a literature review of empirical studies on gamification. In *Proceedings of the 47th Hawaii International Conference on System Sciences*, pages 3025–3034, 2014.
- [9] Andrew Kane. pgvector: Vector similarity search for postgresql. <https://github.com/pgvector/pgvector>, 2026. Accessed 25 September 2026.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

- [11] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026. Accessed 25 September 2026.
- [12] Prometheus Authors. Prometheus documentation. <https://prometheus.io/docs/introduction/overview/>, 2026. Accessed 25 September 2026.
- [13] Redgate. Flyway documentation. <https://documentation.red-gate.com/flyway>, 2026. Accessed 25 September 2026.
- [14] Redis. Redis documentation. <https://redis.io/docs/latest/>, 2026. Accessed 25 September 2026.
- [15] Square. Retrofit: A type-safe http client for android and java. <https://square.github.io/retrofit/>, 2026. Accessed 25 September 2026.
- [16] Yao-Ting Sung, Kuo-En Chang, and Tzu-Chien Liu. The effects of integrating mobile devices with teaching and learning on students' learning performance: A meta-analysis and research synthesis. *Computers & Education*, 94:252–275, 2016.